THE DEVELOPER'S GUIDE TO AI

From Prompts to Agents

by Jacob Orshalick, Jerry M. Reghunadh, and Danny Thompson

no starch press®

San Francisco

THE DEVELOPER'S GUIDE TO AI. Copyright © 2026 by Jacob Orshalick, Jerry M. Reghunadh, and Danny Thompson.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

Printed in the United States of America

First printing

30 29 28 27 26 1 2 3 4 5

ISBN-13: 978-1-7185-0476-9 (print)
ISBN-13: 978-1-7185-0477-6 (ebook)

Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Manager: Sabrina Plomitallo-González
Production Editor: Jennifer Kepler
Developmental Editor: Sierra Fong
Cover Illustrator: Rob Fiore
Interior Design: Octopod Studios
Technical Reviewer: Nikhil Kapoor
Copyeditor: Sharon Wilkey
Proofreader: Gina Linko
Indexer: BIM Creatives, LLC

Library of Congress Control Number: 2026933668

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com. The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an "As Is" basis, without warranty. While every precaution has been taken in the preparation of this work, neither the authors nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

[S]

To my incredible and always encouraging wife, Jennifer. Thank you for all your support and for putting up with me along the way. I couldn't have done this without you. To my wonderful kids, Talia and Kaleb. You continue to impress and inspire me with your dedication and determination in everything you do. And to my parents. Your unwavering love and support have shaped every word on these pages.
—Jacob
(Philippians 4:13)

To my wife, Bhavna; my lovely daughters, Emma and Erin;
and the joy of our life, Joey (woof!).
To my parents, who gave me all the opportunity to succeed.
—Jerry
(Bhagavad Gita 2.47)

To the one who taught me that wisdom isn't about knowing all the answers but asking better questions: my wife. This book is a result of the questions that you asked me to keep chasing.
To the one who helped me learn that curiosity does not have to be tied to an explicit purpose or end goal, but can be used to learn without a specific objective: my son.
Lastly, to the one who heard "no" more often than "yes" but refused to allow it to shake your path, and who is the reason I am in the current place in my career because without your determination, this would never have happened: me.
—Danny

About the Authors

Jacob Orshalick's software development career spans over two decades. After spending several years in academia as a PhD research assistant, he realized his passion was building software that solved real-world problems. Since then, he has spent over 18 years as an independent consultant building software solutions for startups, Fortune 500 companies, and organizations of all sizes in between.

A seasoned advisor and technologist, Jacob has consistently built solutions that solve his clients' most intricate software challenges. He has led state-of-the-art development projects, directed technology refresh initiatives, and revitalized projects on the brink of failure. His career has remained focused on helping his clients harness the latest technology to move their businesses forward.

Jacob is a co-author of *Seam Framework: Experience the Evolution of Java EE* and the author of numerous published articles, and he has won awards for his open source contributions. An accomplished speaker, he regularly presents at conferences and user groups to contribute his knowledge and expertise back to the community. You can explore his writing and learn about his company at *https://focus.dev*.

Jerry M. Reghunadh is a senior director with a career spanning over two decades. He is driven by an insatiable curiosity and a profound passion for learning. He approaches every new challenge with a childlike vigor and a "just want to know" attitude, a quality that has enabled him to master highly technical concepts, break them into manageable insights, and apply them successfully in both personal projects and professional endeavors.

His career journey began as a QA engineer, where he built and maintained multiple automation suites, eventually leading him to his current role as a senior director instrumental in building innovative products and architecting complex solutions for customers.

Jerry is a firm believer in the power of shared knowledge and frequently speaks at conferences, both locally and internationally.

At heart, Jerry is an engineer who thrives on continuous learning, experimentation, and problem-solving, whether it's tackling an old challenge or a brand-new one. He holds a unique perspective that writing quality code that performs optimally on even bare-bones hardware is an art form.

In his spare time, he's often found "fixing" things around the house—much to the playful anguish of his wife and two daughters—and he's an avid Formula 1 fan, passionately following the sport. You can explore his thoughts and experiments at *https://jerrymannel.me*.

Danny Thompson's journey is a testament to transformation, passion, and a relentless drive to make technology accessible to all. Starting out frying chicken at a gas station, Danny discovered an early passion for coding—a spark that ignited a career marked by continuous learning and innovation. His path led him to work with Fortune 500 companies, ultimately paving the way to his role as director of technology, in which he leverages years of hands-on experience to simplify even the most complex technical challenges.

Known for his unique ability to break difficult subjects into clear, actionable insights, Danny has made a lasting impact as an educator and leader in the tech community. He hosts the globally top-ranked *The Programming Podcast*, inspiring a worldwide audience with engaging discussions and expert insights. In addition, his commitment to community building shines through his work as a conference organizer—most notably with the Commit Your Code Conference for Charity—and as the driving force behind the Dallas Software Developers group meetup.

As an accomplished instructor, international corporate speaker, and keynote presenter, Danny continues to empower and educate the next generation of developers. His career is not just a story of personal success but also a beacon for anyone striving to transform challenges into opportunities through hard work, creativity, and a passion for technology.

About the Technical Reviewer

Nikhil Kapoor has over 16 years of experience in the fields of artificial intelligence and machine learning, where he has designed and implemented advanced models, intelligent platforms, and data-driven solutions at scale. His expertise extends to architecting end-to-end platforms and distributed applications within a leading cloud service provider, integrating AI/ML capabilities with cloud-native technologies to drive innovation. He has applied these skills across diverse sectors—including finance, investment management, audit, and telecom R&D—delivering intelligent, scalable, and production-ready systems. In addition to technical depth, he also has led teams through the full software development life cycle.

BRIEF CONTENTS

Acknowledgments xix
Authors' Note xxi
Preface xxiii
Introduction xxvii

PART I: GETTING STARTED WITH AI

Chapter 1: Understanding Large Language Models 3
Chapter 2: Building Your First LLM-Powered Application 15
Chapter 3: Python Essentials for LLMs and APIs 29

PART II: PROMPT ENGINEERING

Chapter 4: Fundamentals of Prompt Engineering 43
Chapter 5: Prompt Engineering Techniques 61
Chapter 6: Prompt Engineering in Code 77

PART III: VECTOR DATABASES AND RAG

Chapter 7: Vector Databases in Practice 127
Chapter 8: Designing a Retrieval-Augmented Generation System 157

PART IV: ADAPTING MODELS TO REAL-WORLD TASKS

Chapter 9: Why and When to Customize a Model 193
Chapter 10: Preparing Data for Fine-Tuning 203
Chapter 11: Fine-Tuning Models in Practice 215

PART V: BUILDING AGENTIC SYSTEMS

Chapter 12: From Workflows to Autonomous Agents 243
Chapter 13: Building an Autonomous Agent 249
Chapter 14: Extending Agents with Tools 255

Afterword 269
Index 271

CONTENTS IN DETAIL

ACKNOWLEDGMENTS **xix**

AUTHORS' NOTE **xxi**

PREFACE **xxiii**

INTRODUCTION **xxvii**

Generative AI and Beyond xxviii
Data Security and Privacy xxix
Technology Choice xxix
Downloading the Example Code xxx
The AI Coding Assistant Safety Recall Notice xxxi
What We'll Cover xxxi

PART I
GETTING STARTED WITH AI

1
UNDERSTANDING LARGE LANGUAGE MODELS **3**

Understanding Text Generation 5
AI vs. Traditional Logic 6
The Versatility of LLMs 7
Understanding the Limitations 7
- Limited Training Knowledge 8
- Hallucinations 9
- Model Bias 9
- Multi-Step Reasoning 9

Improving LLM Responses 10
- Prompt Engineering 10
- Context Engineering 11
- Fine-Tuning 11
- Custom Models 11

Building Autonomous Systems 11
Plugging LLMs into Your Code 12
Summary 13

2
BUILDING YOUR FIRST LLM-POWERED APPLICATION 15

Calling an LLM by Using an API . . . 16
Verify System Requirements . . . 16
Install Ollama and Run a Local LLM . . . 17
Send a Prompt to the Local LLM . . . 18
Install Node.js . . . 19
Initialize Your Project . . . 19
Install Node.js Packages . . . 19
Create the Server File . . . 20
Launch Your Server and Test . . . 22
Streaming Your LLM Responses . . . 22
Stream an LLM Response with Ollama . . . 23
Stream a REST Response with Express . . . 23
Install the CORS Middleware Module . . . 24
Create the Entire Server File with Express . . . 25
Launch Your Server and Test . . . 26
Stream to the UI with Fetch . . . 26
Decode the Response Chunks and Render . . . 27
Summary . . . 28
Exercises . . . 28

3
PYTHON ESSENTIALS FOR LLMS AND APIS 29

Installing Python and Libraries . . . 30
Calling an LLM via an API . . . 31
Create the FastAPI Server . . . 31
Understand the Components . . . 32
Set Up the FastAPI Server . . . 33
Send a Prompt to the Local LLM . . . 34
Launch Your Server and Test . . . 35
Summary . . . 36
Exercises . . . 37

PART II
PROMPT ENGINEERING

4
FUNDAMENTALS OF PROMPT ENGINEERING 43

Programming vs. Prompting . . . 44
A Clear Use Case for an LLM . . . 45

Evaluating Your Use Case 48
Prompting Basics 48
Prompt Elements 50
Context Windows 51
Tokenization 52
Avoiding Runaway Costs 55
Accuracy and Performance 56
Choosing a Model 57
Summary 60
Exercises 60

5
PROMPT ENGINEERING TECHNIQUES 61

Instruction Prompting 62
    Data Extraction 62
    Personally Identifiable Information Removal 63
    Synthetic Data Generation 65
Persona Prompting 66
Zero-, One-, and Few-Shot Prompts 68
Chain-of-Thought Prompting 72
Prompt Chaining 73
Prompt Tips and Best Practices 74
    Focus on the Positive, Avoid the Negative 74
    Collaborate with the Model on Your Prompt 74
    Treat Your Prompts Like Code 74
    Experiment with Prompt Variations 74
Summary 75
Exercises 76

6
PROMPT ENGINEERING IN CODE 77

Choosing a Library 78
Connecting to a Hosted API 80
LLM Output Configuration 81
    Output Length 81
    Sampling Controls 83
Prompt Templates 85
    Creating a Template with Jinja 85
    Separating Prompts and Code with Jinja 88
    Using Dynamic Prompt Templates with Jinja 89
Messages and Roles 91
    Understanding Chat APIs 92
    Including System Messages 95
    Creating a Type for Messages and Roles 96

Conversation History 96
Maintaining Conversation History on the Client 97
Trimming the Conversation History 100
Summarizing the Conversation History 102
Choosing Conversation History Options 105
Structured Output 107
Defining a JSON Schema with Pydantic 107
Using Instructor 113
Extracting Data with Instructor 118
Prompting in Production 121
Request Retries 121
Guardrails and Injection 121
Token Counts and Usage Costs 123
Summary 123
Exercises 124

PART III
VECTOR DATABASES AND RAG

7
VECTOR DATABASES IN PRACTICE **127**

Helping Machines Understand Our World 128
Powering a Better Search Experience 129
Understanding Vector Embeddings 131
Choosing an Embedding Model 137
Dissecting Model Names 138
How Vector Database Retrieval Works 140
Using a Vector Database for Meaningful Search Results 142
Creating the Product Collection 142
Inserting Products into the Vector Database 143
Querying the Vector Database 146
Using a Vector Database to Make Recommendations 147
Defining the Data Classes 148
Retrieving the Recommendations 150
Identifying the Most Relevant Results 151
Filtering on Metadata 152
Summary 154
Exercises 154

8
DESIGNING A RETRIEVAL-AUGMENTED GENERATION SYSTEM **157**

What Is Retrieval-Augmented Generation? 158
Connecting the Knowledge Base 160
Using Context Retrieval with a Vector Database 162

Understanding the RAG Pipeline 162
Setting Up the Environment 164
Chunking and Loading a Vector Database 165
Understanding Chunking Basics 165
Choosing a Chunking Strategy 167
Chunking the Documentation 169
Loading Chunks into the Vector Database 171
Completing the RAG Pipeline with an LLM 173
Creating the Prompt Templates 174
Prompting an LLM with Context 175
Creating the Service API 177
Improving Trust with Citations 178
Advanced RAG Topics 183
Alternative Indexing and Retrieval Strategies 183
Architectural Variants 185
Production-Level Refinements 186
Data Quality and Security 188
Summary 189
Exercises 190

PART IV
ADAPTING MODELS TO REAL-WORLD TASKS

9
WHY AND WHEN TO CUSTOMIZE A MODEL 193

Building a New Model vs. Fine-Tuning 195
Fine-Tuning Strategies 197
Self-Supervised Fine-Tuning 198
Supervised Fine-Tuning 198
Reinforcement Learning Fine-Tuning 198
Fine-Tuning Methods 199
No-Code Fine-Tuning 199
Technical Fine-Tuning 201
The Fine-Tuning Process 201
Summary 202

10
PREPARING DATA FOR FINE-TUNING 203

Gathering the Raw Data 204
Structuring the Raw Data 205
Encoding the Labels 205
Splitting the Data 206

Creating a Labeled Dataset 208
Creating a Dataset 209
Creating the Train, Validate, and Test Datasets 210
Combining the Datasets 210
Verifying the Dataset 212
Fixing the Label 213
Summary 214
Exercises 214

11
FINE-TUNING MODELS IN PRACTICE 215

Choosing Models for Fine-Tuning 216
Establishing a Baseline 216
Fine-Tuning a Classification Model 217
Evaluating the Fine-Tuned Classification Model 221
Fine-Tuning an LLM 222
Why Not Prompt Engineering? 222
Introduction to PEFT and LoRA 224
Regular Fine-Tuning vs. PEFT 224
LoRA 225
Prompting an LLM 226
Understanding Instruction Templates 226
Customizing Generated Response Format 230
Controlling Where Generation Starts 231
Building the Dataset 231
Training the LLM Using LoRA 234
Reducing Parameters 234
Configuring LoRA 235
Training 238
Evaluating the Fine-Tuned Model 239
Summary 239
Exercises 240

PART V
BUILDING AGENTIC SYSTEMS

12
FROM WORKFLOWS TO AUTONOMOUS AGENTS 243

What Are AI Agents? 244
Why Do We Need Agents? 244
Traditional Workflows 245
AI Workflow 245
Agentic Workflow 246
Summary 247

13
BUILDING AN AUTONOMOUS AGENT **249**

Setting Up Your Agent Development Environment 250
Building Your First Agent 251
Agents and Prompt Engineering 252
Summary 253
Exercises 253

14
EXTENDING AGENTS WITH TOOLS **255**

Building a Tool 256
Writing Effective Tools 257
Building Reusable Tools 257
Model Context Protocol 258
    MCP Concepts 258
    MCP Components 259
    MCP Transport Protocols 259
Building an MCP Server 259
Your First MCP Server 260
    Building a Special Calculator 260
    Defining Custom Tools 261
    Running the MCP Server 261
    Testing with Postman 262
    Integrating with Claude Desktop 263
MCP Transports 265
Building Tools with Complex Schemas 265
Summary 267
Exercises 267

AFTERWORD **269**

INDEX **271**

ACKNOWLEDGMENTS

This book would not have been possible without the immense dedication and support of a great many people. It is a true testament to their hard work and belief in this project.

Our deepest gratitude goes to the entire team at No Starch Press. We are especially thankful to Bill Pollock for his vision, Jill Franklin for her unwavering support, and Eva Morrow, Sierra Godfrey Fong, Miles Bond, and the rest of the team for their meticulous guidance throughout the entire publishing process.

We are incredibly grateful to the brilliant minds who dedicated their time and expertise to review this book. Their critical insights, constructive suggestions, and keen attention to detail were instrumental in refining the content and ensuring its clarity and accuracy.

We also extend our sincere appreciation to Erik Weibust, Srinivasan Raguraman, and Preet Katari. Their expertise was crucial in verifying the accuracy of our code examples and the technical concepts presented, ensuring the content is both reliable and practical for our readers.

A special thank-you to our early reviewers: Joseph Evans, Edward Kataika, David Archuleta Jr., Patricio Horn, Marco Chiesa, Raghavendra Sagar Thorahatula, Saurabh Gupta, Abim Shrestha, David Kea Jr., Sean Campbell, Alex Harris, Melisa Im, Linda Thompson, Daniel Cassidy, Sara Baqla, Joseph Evans, and Andrew Teesdale Jr.

We're also very grateful to our talented illustrator, Aabha Kalé, for creating the amazing cover art for the Commit Your Code early edition.

This journey began with the support of a few key communities. A special shout-out to the Dallas Software Developers community and the Commit Your Code Conference for being the catalysts that sparked this entire endeavor.

Finally, we want to thank our families and friends for their endless encouragement, understanding, and patience during the long hours spent bringing this book to fruition. Your support means the world to us. And to you, the reader, we hope this book empowers your journey into the exciting world of AI. Your engagement is the ultimate reward for our efforts.

AUTHORS' NOTE

The world of AI and its tools are moving at breakneck speed. We've taken the utmost care to make sure everything in these pages is as up to date as possible, but when you're working at the cutting edge, the ground tends to shift.

Even if a specific library updates or a code snippet changes, the core concepts we've covered are here to stay.

PREFACE

A few years into my computer science degree at the University of Texas at Dallas (UTD), I signed up for an internship program. UTD is situated in the middle of what used to be known as the telecom corridor in Richardson, Texas. A friend told me the telecom companies were paying computer science interns much more than I could make as a waiter. So I could code *and* start making money doing it! I was in.

I was hired as part of a company's Intranet Team, which sounds a lot more exciting than it turned out to be. We were responsible for keeping the company's internal website up to date. This website was a collection of manually updated HTML pages that summarized and linked to resources, company press releases, announcements, and help pages. I was hired to handle the press releases.

The process was simple. The marketing team would send me a press release. I would duplicate an existing HTML press release page, replace the contents, tweak where necessary, and upload the new HTML file. This got repetitive and boring really fast. So what does any good programmer do? Automate.

I built a form within the intranet website that allowed the marketing team to submit the press release text. A backend job would then wrap the press release with the necessary HTML and add it to the intranet website automatically. When my team members saw this, they wanted me to apply this approach to the rest of the website. Of course! That's way more exciting than copying and editing HTML. I didn't know I was building a content management system. This was a year before the first release of WordPress (really dating myself now).

By the end of my internship, the team I was hired to work with needed new assignments. The business teams could now update the entire intranet website on their own. I found out months later the entire Intranet Team was laid off. The company no longer had a need for their skillset because their jobs had been automated. I felt guilty, but it taught me a valuable lesson early in my career: If you are going to make it in this field, you need to stay current with technology. Otherwise, you may be the one automated out of a job.

Fast-forward to early 2023: ChatGPT had recently been released, tech firms started laying off thousands, and rumors swirled that artificial intelligence (AI) would soon replace developers. It looked like the writing was on the wall and it was time to pivot to data science! Well, it turned out that large language models (LLMs) write buggy code and tech firms had simply over-hired, but who knew at the time?

So I began my AI journey. It was daunting. I had studied AI in college, but after 20 years, the world of AI looked completely different. I needed to start from scratch. I needed to understand the concepts!

I ordered a two-inch-thick textbook that seemingly provided the entire history of AI. It walked through the early beginnings, game theory, statistical models, and eventually neural networks. It was full of mathematics, Python recipes, and Jupyter notebooks.

I nearly gave up. AI seemed like nothing but statistics and linear algebra, my least favorite courses in college. I started to imagine I would face the same fate as the Intranet Team! Then, as I started researching LLMs and OpenAI, I found a glimmer of hope: application programming interfaces (APIs) and software development kits (SDKs). This led me to solutions that researchers and startups were using to combine pretrained AI models with their own data—without training! No statistics. No linear algebra. I could combine these APIs with database and search techniques to build truly intelligent systems. As with anything, once I understood these LLMs and how to use them, the fear vanished.

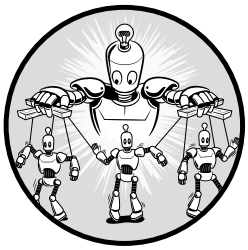
Which brings me to the goal of this book. This book is written by developers for developers. It's the road map I wish I'd had when I started learning. It will teach you the concepts needed to work with pretrained AI models, but in a way that makes sense to a developer. Instead of fearing the machine, you'll be able to feed it with tools you already know and love. All you need to get started is basic JavaScript knowledge, curiosity, and a willingness to learn.

You'll find out how pretrained LLMs enable the building of intelligent systems with nothing but tools you're already familiar with: APIs, SDKs, and databases. Not only that, all the examples are based on real-world solutions, so you'll understand how actual companies are already using this technology. We'll dive deep into how to customize pretrained AI models (LLMs and beyond) to suit your needs, create your own autonomous agents, and even help you learn Python along the way. But we'll teach you everything from a developer's perspective. No data science necessary.

By the end of this book, you'll have the conceptual knowledge and the code necessary to catch the AI wave today. So come on. Let's get your AI learning journey started. It's never been easier.

Jacob Orshalick

INTRODUCTION



You don't need to build AI models to create something remarkable with them! This book is for developers like you. It's built on a simple, powerful idea: We aren't AI architects, who speak in mathematics. We are AI chefs, and we speak code. We don't need to build the oven from scratch. We need the recipes to piece together *pretrained* AI models, APIs, and databases to build something incredible. This book is *that* recipe guide.

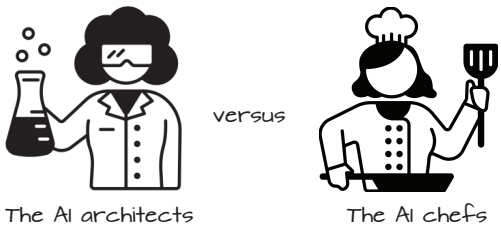
You'll follow the journey of a startup team as they face the *exact* problems you're likely facing right now. Their journey starts with a rushed solution: using an API call to an AI model to handle an avalanche of customer support. But what happens when that first call works—sort of—but the AI model knows nothing about the actual product?

How much more effective would your systems be if an AI model had "memory" of your *own* internal knowledge and conversations? You could feed an AI model everything it needs to know to answer not only support questions but also questions about your documentation, internal documents, data, and more. What would it mean for your team if you could avoid the sticker shock of that first bill for sending massive, token-heavy

API prompts? Your team could engineer prompts that contain *only* the necessary context by using a smarter solution like retrieval-augmented generation (RAG).

What if you could solve the problem of your overflowing inbox? You could *teach* an AI model a new, specialized skill, like fine-tuning it to classify your *exact* email types: Sales Lead, Support Query, or Junk. How much more valuable would your systems be if Sales Lead profiles could be autonomously extracted from email and added to a database? An AI model could help you pull important data from not only email but also your documents, contracts, tickets, logs, and more so you could unlock insights that were previously trapped in cluttered folders and inboxes.

Finally, think about what your real bottleneck is. Is it sorting the mail or *acting* on it? How much more valuable would your team be if the AI could autonomously reason, plan, and use tools to handle an entire workflow, transforming it from a mail sorter into a true digital collaborator?



This journey is for the software engineer who has no desire to get a master's in machine learning but has a deep desire to stay relevant. It's for the manager who knows their team is capable of more, if they could just be freed from the manual tasks that drain their creativity. It's for the junior developer who sees everything changing around them and is trying to make sense of all this new technology.

Our job as developers is to understand the tools available to us so we can solve real problems. This book will add AI to your toolkit so you can solve problems you never could before. You'll learn everything you need so *you* can go from writing prompts to building agents.

Generative AI and Beyond

The release of ChatGPT vaulted AI into mainstream culture, as generative AI was suddenly accessible to everyone. After experimenting with ChatGPT, we were all astonished. It was pure magic. And regardless of our technical abilities, we all had the same questions: Was this computer actually thinking? Was it some superhuman intelligence? Were we witnessing the rise of the machines?

OpenAI had us believing that artificial general intelligence (AGI) was on the horizon. It wasn't, and *probably* still isn't. ChatGPT simply gave the world its first glimpse of generative AI. It provided an easy-to-use interface to the true marvel that lies beneath: the large language model (LLM).

LLMs are a great place to begin our AI journey. These popular generative AI models are general-purpose. They're pretrained to accept input and generate output we can all understand: natural language. We've all used LLMs, and in this book they'll help us introduce AI concepts and tools you'll need.

As you progress through the book and your conceptual knowledge grows, we'll introduce additional types of pretrained AI models you can add to your toolkit. We'll even show you how to use these models to build autonomous systems. By the end of the book, you'll be an accomplished AI chef ready to build real-world solutions.

Data Security and Privacy

Before sending confidential data to an LLM (or any AI model), you need to understand the risks. It is important for companies to know how their proprietary data will be used by LLM providers.

OpenAI's data privacy policy for personal ChatGPT workspaces states that chat conversations may be used to improve its models unless you explicitly opt out. The platform's usage guide also cautions users against sharing sensitive information during any conversation. Samsung learned this the hard way.

The Economist Korea reported three incidents in which Samsung Electronics employees accidentally exposed sensitive information to ChatGPT. In one case, an employee inserted confidential source code into a chat to debug errors. Another employee shared code with ChatGPT to request optimization. In a third instance, an employee uploaded a meeting recording to have it transcribed into presentation notes. After discovering the leak, Samsung sent a warning to its employees and restricted access to ChatGPT.

Before sending confidential information to any externally hosted LLM, make sure you understand its data security and privacy agreements. We will discuss this in more detail in Chapter 4 to provide some guidance on data security and privacy as it relates to LLMs.

Technology Choice

Our goal in this book is not to sell you on any particular technology choice. We aim to give you the knowledge needed to become an AI chef so you can build incredible systems with AI models.

JavaScript is almost universal among developers, making it the perfect place to start. We'll use it to explain many core AI concepts. If you can read or write simple JavaScript code, you'll have everything you need to get started.

We'll then ease you into Python so we can dive deeper into advanced AI topics. Python offers the most mature ecosystem for AI development, with a rich set of libraries and tools that make it ideal for demonstrating complex implementations.

The examples use a variety of pretrained AI models (for example, GPT, Llama, and Gemini models) from various providers (for example, OpenAI,

Meta, Google, and Hugging Face). We'll provide tips on how to select a model along the way.

We will also introduce you to a variety of tools, libraries, and frameworks to help explain architecture and best practices. If you decide to choose one of these technologies, great! The important point is that you understand the concepts first. In this way, you will know when and why a technology is the right choice.

NOTE *We'll reference specific versions of pretrained AI models, tools, libraries, and frameworks throughout the book. We chose these versions intentionally. Specific versions ensure accuracy in code examples (as when using SDKs) and when explaining fundamental concepts (like training cutoff dates and context window sizes). If you're using a different version or even a different tool, library, framework, or model, not to worry. The concepts and fundamentals remain the same.*

Downloading the Example Code

The world of AI is changing fast. We're dedicated to keeping the book examples up to date. You can always access the latest and greatest code from the GitHub repository at *https://github.com/jorshali/developers-guide-to-ai*.

We have also added (and will continue to add) examples beyond those in this book. We encourage you to fork the repository and experiment on your own. If you have an example you think is worth adding, let us know! We love pull requests.

We show a lot of code snippets throughout the book. Keep in mind that we often include only the code snippet relevant to a concept being discussed. You will see the following conventions:

`--snip--` Indicates code before or after the snippet that is not shown

`# ... ...` Indicates code within a snippet that is not shown

For library imports, we show only those that are relevant to the code snippet presented. If the code snippet is a continuation, we may exclude a library import indicated previously. Keep in mind that if you want to see an entire file, you can access the complete code in the GitHub repository.

We also include a lot of print/log statements in the examples. Printing out results, object structures, and the execution of important logical steps is an extremely useful way to understand and debug code. If an example isn't making sense, add your own print statements! You'll be amazed at how much this can help.

To make setup simple, we've included the necessary dependency files: *package.json* for Node.js and *requirements.txt* for Python. Each project also has an accompanying *README.md* file that explains what the code does and how to get it running.

The AI Coding Assistant Safety Recall Notice

We can't write an AI book for developers without mentioning AI coding assistants. They can accelerate your productivity, but they still make *a lot* of mistakes. You wouldn't drive a car without a seatbelt, would you? Don't be a victim! Buckle up for safety first:

- Understand before generating. Never have the AI assistant generate code you don't understand. You always need the knowledge necessary to know whether the code is correct.
- Use version control. Your project should be under version control before using AI assistance. You need the ability to back out any code changes it makes.
- Implement incremental changes. Have the assistant make small, incremental changes that can be easily reviewed. If you've ever done a code review, you know how hard it is to review large commits.
- Commit frequently. Using version control doesn't help unless you commit your code before the AI assistant makes changes. This gives you a way to fall back to a working version.
- Diff is your friend. Since you're using version control and you commit frequently, you can easily see what the AI changed with `git diff` (or insert your version control `diff` command).

These are really just good software engineering practices that we should all be following. Okay, you've been warned.

Here are some popular AI coding assistants you can try:

- Claude Code (*https://claude.com/product/claude-code*)
- GitHub Copilot (*https://github.com/features/copilot*)
- Cursor (*https://www.cursor.com*)
- Windsurf (*https://windsurf.com*)

What We'll Cover

This book is organized into five parts. Part I focuses on using LLMs as is. This part simply requires basic JavaScript knowledge and a willingness to learn. We'll set the stage by introducing you to LLMs through a use case everyone can relate to: an AI support agent. After a quick introduction to Python, we'll start diving deeper. The chapters in this part are as follows:

Chapter 1: Understanding Large Language Models We'll explain what LLMs are in plain terms, why they're powerful for automating knowledge work, the limitations to plan for, and the practical levers teams can use to reliably integrate LLMs into real solutions.

Chapter 2: Building Your First LLM-Powered Application You'll get your first experience coding an AI-powered solution as you build a simple, real-world chat-style app using JavaScript and an LLM running on your machine.

Chapter 3: Python Essentials for LLMs and APIs We'll get you productive in Python by re-creating the chat-style app while introducing the language's foundational syntax, concepts, and libraries that you'll need for the rest of the book's examples.

Part II is a practical guide to prompt engineering. You will learn a mental framework and techniques for creating effective prompts. You will also learn about the libraries and frameworks available to integrate LLMs in your applications. Examples and code will guide you along the way so you will be comfortable instructing LLMs to do your bidding. The chapters in this part are as follows:

Chapter 4: Fundamentals of Prompt Engineering You'll explore the types of real-world problems LLMs solve, the building blocks of a quality prompt, and the constraints that must be managed to keep your AI features accurate, fast, and cost-effective.

Chapter 5: Prompt Engineering Techniques You'll learn how to get more reliable, business-ready results from LLMs by using proven prompting patterns, techniques, and best practices.

Chapter 6: Prompt Engineering in Code You'll see how to turn those quality prompts into reliable, production-ready features by choosing the right LLM libraries and settings, applying proven coding patterns, and handling real-world considerations so your first deployment is intentional, not a gamble.

Part III introduces vector databases and the real-world problems they solve. You will learn about RAG and will use everything you've learned so far to build an AI support agent. We will then introduce you to advanced concepts needed to build a production-level RAG solution. The chapters in this part are as follows:

Chapter 7: Vector Databases in Practice We'll expand your knowledge of pretrained AI models beyond LLMs by showing how businesses can use embedding models and vector databases to power smarter search and personalization, like finding the right products or recommending next-best items.

Chapter 8: Designing a Retrieval-Augmented Generation System You'll use RAG to transform an LLM into an expert on your private data, moving you beyond generic responses to accurate, business-specific answers that drive real productivity from your existing knowledge.

Part IV dives into customizing pretrained AI models and LLMs to suit specific needs. You will learn to fine-tune an LLM to solve specific problems with a high degree of accuracy. The chapters in this part are as follows:

Chapter 9: Why and When to Customize a Model We'll explore why and when to move beyond generic pretrained models. We'll compare the efficiency of fine-tuning against building from scratch and introduce the core strategies and methods used to specialize AI for niche, real-world tasks.

Chapter 10: Preparing Data for Fine-Tuning You'll see how to turn data into a high-quality dataset by curating, labeling, and strategically splitting your data for fine-tuning success.

Chapter 11: Fine-Tuning Models in Practice We'll bridge the gap between theory and code, walking through the end-to-end technical pipeline to transform general-purpose models into specialized, fine-tuned experts.

Part V takes the content you've learned so far and applies it to building autonomous systems with agentic AI. You will learn how to use LLMs to understand a user's goals and the context of a problem. You'll then see how agents can take action to achieve those goals, learning and adapting based on new information. The chapters in this part are as follows:

Chapter 12: From Workflows to Autonomous Agents This chapter marks the transition from reactive AI that simply "thinks" and "responds" to proactive, autonomous agents that "act" and "solve," moving us beyond simple chat windows and into the world of goal-oriented problem-solving.

Chapter 13: Building an Autonomous Agent You'll take your first practical steps toward building AI agents, focusing on a customizable, code-based approach. You'll then build your very first agent.

Chapter 14: Extending Agents with Tools You'll transition from understanding AI as a "thinker" to building AI as a "doer" by exploring how tools and the Model Context Protocol help agents achieve true agency.

What would it mean for your next big project if you, or your team, could turn that chaotic inbox from a graveyard for ideas into an autonomous engine for growth? Let's find out together as we get started!

PART I

GETTING STARTED WITH AI

The Startup Team Begins Their AI Journey

Built on sleepless nights, maxed-out credit cards, and pure stubbornness, the product was ready—barely. But it had an incredible buzz. So when the day of the tech conference came, the goal was simple: find just *one* customer willing to believe. The team, exhausted and anxious, hardly dared to hope.

That's when word started spreading through the small expo hall. First a whisper, then a fire. Demo after demo, vendor after vendor, company after company came through, and *they loved it*. People lined up. Cards exchanged hands. By the end of the day, the dream seemed finally in reach.

But success, as it turned out, came with a cruel twist. The team called it the buzz that broke the bank.

The contracts that the vendors and companies signed included net 90 or even net 120 payment terms. But who paid attention to terms? Contracts were in hand! Only later did the team realize what this truly meant: no money for three to four months. Meanwhile, bills were due *now*. Every server fee, every meal, every subscription drained the bank account. The pipeline looked full on paper but empty in the bank.

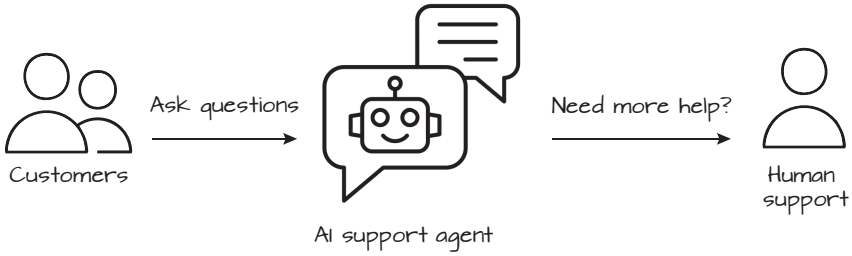
And the bigger problem? Growth brought an avalanche of customer support requests. Endless emails repeated the same five questions again and again. The same simple support conversations were happening every hour of every day. The team had no time left to ship new features or to create a strategy. Just an endless loop of "How do I . . . ?," "Can you help me with . . . ?," and "Where do I find . . . ?"

Increasing the team's headcount was out of the question. The math was brutal. Funding a new hire on top of net 120 payments would bury the company before the invoices ever cleared. One afternoon, while the team was

exhausted and drowning in a sea of half-finished support replies, a friend suggested meeting up for coffee. Somewhere between the third refill and the cold dregs of caffeine, the friend said, almost off-handedly, "You know, if most of your questions are repetitive, couldn't you just train something to answer them for you?"

The idea hit like lightning. Maybe the problem wasn't growth. Maybe the problem was answering every question manually, as if it were still day one. Maybe there was another way. What if the team used an AI support agent?

The team had seen AI support agents before in customer support chats with companies they interacted with every day. The agents would attempt to field questions before a human got involved.



These AI support agents were much more than just a glorified FAQ. They were smart. Some seemed almost human in their responses. Others could even perform tasks, like providing a return label or scheduling an appointment. They were the first automated line of defense. If they couldn't solve a problem, a human could always jump in.

The team knew that training an AI model was out of the question. Training would require a lot more data than they had (they were just a startup), and they didn't have any AI expertise (they were developers, not data scientists!). But maybe training wasn't necessary.

After some research, the team learned that other companies were using large language models (LLMs) without training to build these AI support agents. Those companies were calling these LLMs through APIs and using SDKs. The team knew how to do that!

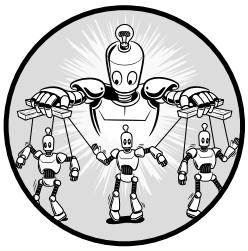
That same day, the startup team was already calling OpenAI's API. The team added a chat interface to their product support page that said, "Need support? For faster service, ask our AI support agent." It worked—sort of.

The LLM could greet users and draft friendly replies, but ask it anything specific like "I'm having password issues. Can you help me log in?," and the response was much less helpful. The AI support agent would either generate generic troubleshooting responses or respond with specific steps for a product that the team had never even heard of!

Calling the LLM was easy, and it was great at generating human-like responses, but it obviously knew nothing about the team's product. How were those other companies using AI models without training? The team had more to learn.

1

UNDERSTANDING LARGE LANGUAGE MODELS



Large language models (LLMs) are very large, deep learning models that are *pretrained* on vast amounts of data. The sheer amount of publicly available data on the internet makes this training possible. Data scientists combine this data with licensed sources and curated datasets to train LLMs to understand patterns and relationships in natural language.

Web pages, books, articles, research papers, code, and more form the foundation for LLM training, illustrating how words connect to create sentences, ideas, and concepts (Figure 1-1). By training AI models on massive amounts of text, researchers discovered that these systems could develop an inherent understanding of language, accumulate knowledge, and even demonstrate elements of reasoning.

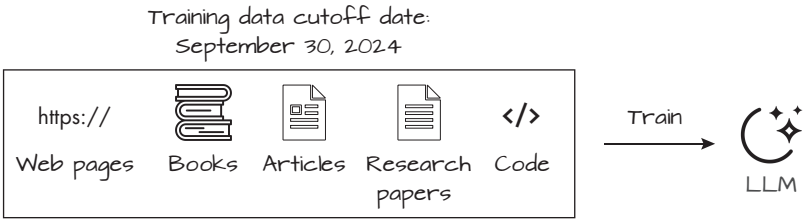


Figure 1-1: LLMs are trained on vast amounts of publicly available and licensed data.

This was the major breakthrough in the field of natural language processing (NLP) when Google researchers released Bidirectional Encoder Representations from Transformers (BERT) in 2018. Table 1-1 shows how the amount of data used to train LLMs has steadily increased over time. Unsurprisingly, research has shown that more training data is better.

Table 1-1: Data Used to Train Some Well-Known LLMs

LLM	Year	Amount of training data	Training data sources
BERT	2018	≈ 3.3 billion words	BookCorpus (800 million words), English Wikipedia (2.5 billion words)
GPT-3	2020	≈ 300 billion tokens	Common Crawl, WebText2, books (multiple sets), Wikipedia
LLaMA (Llama 1)	2023	≈ 1–1.4 trillion tokens	Publicly available web data and curated corpora (Common Crawl, GitHub, Wikipedia, books, and so on)

The computing costs to train your own LLM are estimated at anywhere from hundreds of thousands to millions of dollars, depending on the size of the model. Fortunately for us, companies are racing to release more powerful proprietary models (these cost money to use) and open weight models (similar to open source) every day.

Thankfully, we are being given access to all these AI models through something all developers know how to use: APIs and SDKs.

THE BERT BREAKTHROUGH

Before Google's BERT, NLP models had difficulty understanding complex sentence structures. Researchers at Google started to experiment with masking random words in text and training a model to guess the missing words. The model learned to pay attention to the entire sentence, looking at the words before and after each blank.

Researchers then swapped the order of random sentences and had the model guess whether the sentences followed one another. With this training, BERT began to handle tasks like answering questions and summarizing information with impressive accuracy. The published results of BERT drove incredible advances in NLP, eventually leading to today's LLMs.

Understanding Text Generation

Anyone who has used ChatGPT knows the inputs and outputs of an LLM. We *prompt* the LLM with questions, instructions, and examples, and it generates a *response* (Figure 1-2). The quality of the response generally depends on the quality of the prompt (see Part II).

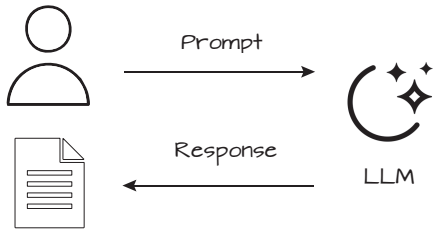


Figure 1-2: Prompts in natural language are the inputs to LLMs.

The model predicts the word most likely to appear next in the sequence, over and over, until the response generation is complete. Each word is referred to as a *token*. Tokens are not exactly words, but it is easy for us to think of them that way (Figure 1-3).

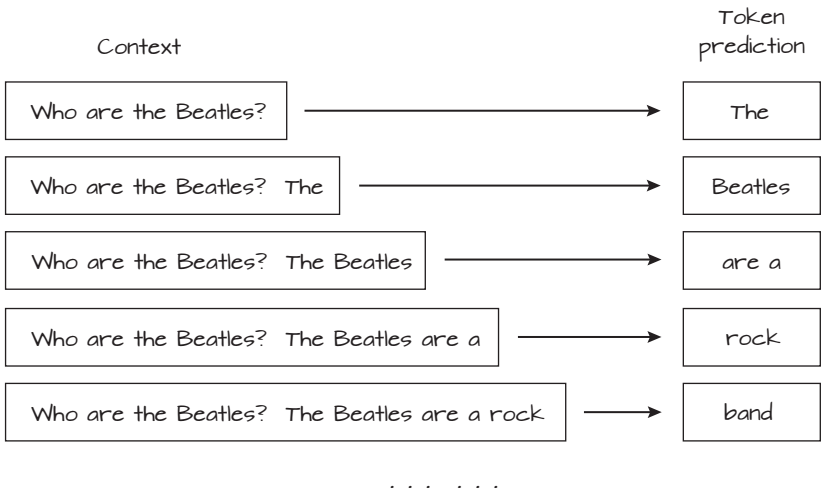


Figure 1-3: LLMs generate responses by predicting a sequence of tokens.

As you can see, the prompt provides the initial *context* for the completion that will be generated by the model (in this case, "Who are the Beatles?"). The model then predicts the *most probable* next token and adds it to the context for the next prediction until a complete response is generated. This step-by-step prediction process is called an *auto-regressive* approach. Each new token is generated based on all the tokens that came before it.

The process of an AI model predicting a response when given new, unseen data (a prompt in the case of an LLM) is referred to as *inference*. It's

why AI models are so useful. Once they've been trained, they can use what they've learned to *infer* a response to new input. This ability to generalize is something that humans easily do but that traditional programming has never been able to achieve.

NOTE *You don't have to understand the architectural underpinnings of LLMs in order to use these models. We'll introduce you to the concepts and technical details you need to build production-ready systems with these models. But if you're curious and really want to understand the transformer architecture that supports nearly all modern LLMs, start with Ashish Vaswani et al.'s paper titled "Attention Is All You Need," which introduces the core ideas behind transformers and remains the foundational reference for anyone looking to dive deeper into how models like GPT and BERT work.*

AI vs. Traditional Logic

Traditional programming is based on Boolean logic. It's deterministic. We know that once we've tested our code, a given input will result in an expected output. Code is reliable and programmed to perform a very specific task.

This can also be a drawback. Our software is limited to the tasks we explicitly program it to accomplish. It can't adapt to new scenarios. Coding our solution correctly often means that an unexpected input results in either some type of validation failure or an error.

AI models are different. They are trained to handle complex rules and are extremely flexible. But this comes at a cost: The results you get with any given input are probabilistic. This means that sending the same prompt to an LLM twice may yield slightly different results.

If you were to ask ChatGPT, "Who are the Beatles?," you would get a response with members of the band, chart-topping songs and albums, and their legacy. If you start a new conversation with ChatGPT and ask the same question, you would get a similar but slightly different response.

This is a key starting point when deciding whether an AI model makes sense for your specific use case. For example, if you're using an LLM, you may need to be able to tolerate variability in the response. Even with a well-constructed prompt, token generation is still based on probability.

Also, remember that testing may prove that your AI model responds correctly 99.9 percent of the time. That may be more accurate than a human at a particular task, but 0.1 percent of the time, AI is still wrong. You need to consider the impact of a wrong answer and how it would be handled.

WHEN WE NEED A HUMAN IN THE LOOP

Always keep in mind that having a human involved in AI decision-making may be desirable or even necessary. This is referred to as having a *human in the loop*. The more critical the decision, the more likely a human should verify an AI response for accuracy.

A life-critical system is an obvious example. If there is even a small probability that a decision could lead to someone's death or severe injury, a human in the loop is almost certainly required.

The Versatility of LLMs

If you've used ChatGPT, you've already seen how flexible these models are. You can ask it to draft an email, summarize a long article, or suggest ideas for your next trip, all with plain-language instructions.

Under the hood, the model wasn't trained for your exact request. First, it's pretrained on diverse text to learn a general, task-agnostic skill: predicting the next token. Then it's instruction-tuned with examples so it learns to follow directions and produce helpful, well-formatted answers when prompted. With a clear prompt (and sometimes a couple of examples), the same model adapts to many tasks, no retraining required.

You can use these models for a wide range of practical tasks:

- Writing and editing (email, drafts, and rewrites)
- Summarizing (articles, reports, and transcripts)
- Classifying and extracting (tagging, pulling key fields)
- Translating (between many languages)
- Reasoning (explaining steps, outlining approaches)
- Coding (snippets, refactors, and tests)
- Performing as chatbots and AI assistants

You can even find LLMs that have been fine-tuned to better handle these and other specific tasks.

If this seems overwhelming, don't worry. We'll walk through examples and prompting techniques for all these use cases and more throughout the book.

Understanding the Limitations

The panacea of AI research is creating a truly general-purpose artificial general intelligence (AGI). LLMs are not AGI, at least not yet. LLMs are incredibly powerful in their ability to understand what we are asking and to generate a response, but they have limits.

The limitations introduced in this section are not intended to be comprehensive. We'll continue to discuss limitations and demonstrate strategies to address them through the book. Just don't fall into the trap of thinking that LLMs will always be your solution. As the saying goes: *When all you have is a hammer, everything starts to look like a nail.*

Our job as developers is to make informed technology choices based on the specific business problem we're solving. We need to understand the tools available, their limitations, and the trade-offs that come with them.

Some problems simply aren't a good fit for LLMs. The same can be said for any technology choice.

Limited Training Knowledge

An LLM's built-in knowledge is limited to the data it was trained on (Figure 1-4). The training data is bound by a cutoff date. The model does not know facts that arose after the cutoff, nor does it know anything about your private data unless you provide that information in the prompt. The model can produce confident but outdated or incomplete answers, especially on recent news, proprietary information, or highly specialized topics.

Figure 1-4: The knowledge of an LLM is limited to its training data.

You can work around this constraint by providing the missing context within your prompt at inference time. This is referred to as *AI grounding*.

In our AI support agent example, the LLM being called had no knowledge of the product because it wasn't trained or fine-tuned with that information. If the team had a knowledge base of how-to support articles, they could include them in the prompt to properly ground the response (Figure 1-5).

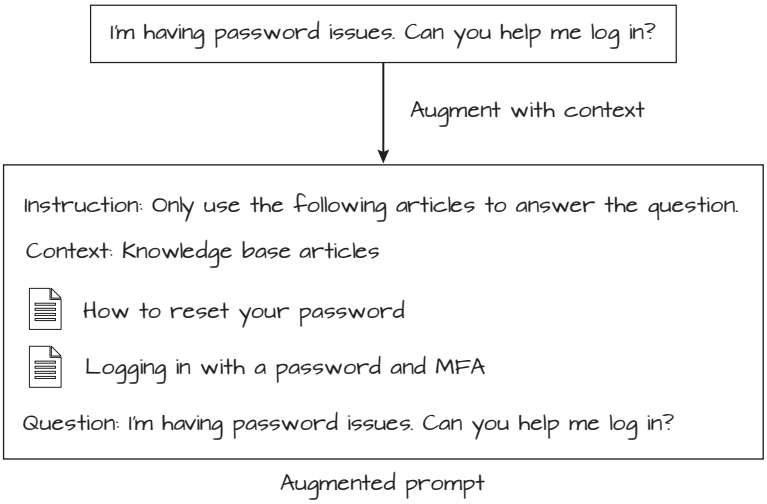


Figure 1-5: The prompt contains the user's question along with how-to articles for context.

In Chapter 8, you'll learn about retrieval-augmented generation (RAG), which is a targeted strategy for supplying only the context that's needed to generate a response.

Hallucinations

If an LLM has not been trained on something you are asking, it may predict a sequence of tokens to answer your question based on patterns it has seen before. The response will seem confident, even if the model has no idea what you are talking about. The LLM may even seem to make up fictitious information! This is referred to as a *hallucination*.

We can all relate. You're having a conversation with someone and think you know exactly what they're talking about. They ask you a question, and you respond confidently with amazing insight. Suddenly, they look at you as if you have three heads. You were so sure, but now you realize you confused fragments of what you thought you knew into something that doesn't quite apply. This has never happened to any of us, right?

Hallucinations can also be caused by the probabilistic nature of LLMs. Remember that each token is predicted based on its probability of occurring next in a sequence of tokens. Even tokens with a high probability of being next could be wrong, depending on the context.

We'll cover a variety of strategies to help prevent hallucinations when we cover prompt engineering in Part II. Just keep in mind that hallucinations are always possible because of the nature of LLMs.

Model Bias

Model bias happens when AI learns patterns from data that already contains human biases or gaps. If the training data leans a certain way (say, with content that is mostly English or from a specific region) or repeats stereotypes, the model can repeat those patterns. It's not the model "choosing" to be unfair; it's simply echoing what it saw.

While the vast amount of data available on the internet makes LLM training possible, we all know that human bias is everywhere. Data scientists do their best to filter out bad training data, but models learn language patterns without moral or ethical considerations.

This means their output can be skewed by their training data. If we recognize bias in our responses, we may need to add filtering, adjust our prompts, or fine-tune the model. We'll discuss this more in later chapters.

Multi-Step Reasoning

Multi-step reasoning is hard for LLMs because they don't "think" through a problem the way people do. They generate the next likely word based on patterns, not a plan. Therefore, LLMs can skip steps, mix up facts, or make confident leaps that look right but aren't.

Errors also compound. If one early step is off, later steps build on that mistake. Long prompts make this worse. Important details can get buried, the model loses track of earlier points, and it fills gaps with guesses instead of asking for missing information.

LLMs also lack reliable self-checking. They don't know when they're wrong, they are sensitive to the way you phrase the question, and they won't

verify math or logic unless you force them to. They don't maintain a true working memory across turns, so context can drift in longer tasks.

We'll discuss some prompting techniques in Chapter 5 that help with handling multi-step reasoning tasks.

Improving LLM Responses

An LLM is essentially a black box. We don't know exactly what data the model has been trained on. For any pretrained model, we have to experiment to determine whether it can provide a useful response as is. As you've already learned, we can influence the accuracy of the response based on a number of factors. The old adage always applies: *Garbage in, garbage out.*

Figure 1-6 shows the core customization strategies aimed at improving model accuracy and how they compare in terms of time, cost, flexibility, and precision.

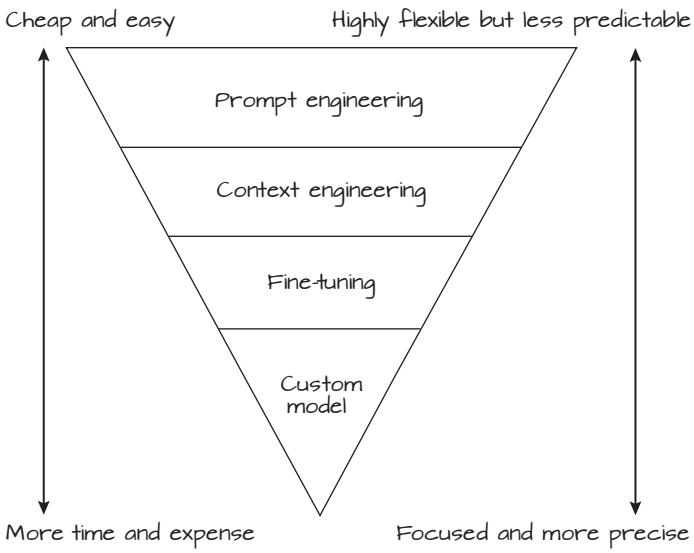


Figure 1-6: The hierarchy of model customization strategies

Throughout the remainder of the book, we'll give you the knowledge you need to progress down the pyramid. The following provides a quick overview of each of these strategies.

Prompt Engineering

Prompt engineering requires creativity and trial and error. The goal is to create prompts that provide context, instructions, and examples to help an LLM understand your intent and generate a desirable response. Part II is dedicated to the art and science of prompt engineering. For now, it's key to understand that your ability to write good prompts will determine your success with LLMs.

It's always easiest (and the most cost-effective) to start with prompt engineering. We simply create prompts and experiment with the accuracy of the LLM response. If the results are promising, but we need more precision in our responses, we can continue down the pyramid.

Context Engineering

Context engineering encompasses prompt engineering along with the system components necessary to efficiently manage context within a prompt. In the words of Andrej Karpathy (former senior director of AI at Tesla and co-founder of OpenAI):

> People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. [But] in every industrial-strength LLM app, context engineering is the delicate art and science of filling the context window with just the right information for the next step.

You'll learn more about context windows and the importance of managing the size of your prompts in Chapter 4. In the simplest terms, the context window defines the limit on the size of a prompt that can be sent to an LLM.

As you'll see, when we're supplying context to an LLM, it's important to include only the facts it needs (snippets from documents, search results, or memory), and put them in a clear order. This becomes even more important in multi-turn scenarios (for example, chatbots or agents), as conversation history can quickly fill the context window.

Fine-Tuning

Even context engineering may not be enough. Sometimes you need the model itself to be fundamentally better at a particular skill or to adopt a specific style that is crucial for your application. This is the realm of fine-tuning: taking an already powerful LLM and training it further on specialized data to mold its core capabilities precisely to your needs. In Part IV, you'll learn everything you need to know to fine-tune a pretrained AI model.

Custom Models

Custom models are beyond the scope of this book. Remember, we're the AI chefs. We use APIs and SDKs to integrate pretrained AI models. We leave the model building to the AI architects. That said, if you want to become an AI architect, this book will give you the knowledge to know where to start.

Building Autonomous Systems

Once you've learned how to bend LLMs to your will, you can use this knowledge to build truly autonomous systems. As we discussed earlier, traditional software systems are programmed to work in a specific way. Given a set of

inputs, the system will do exactly what it was programmed to do. Any new workflow requires you to change the system.

Agentic AI systems are different. These systems can operate autonomously, making decisions, taking actions, and adjusting their behavior over time to achieve specific goals. They do this by coordinating tools, APIs, or even other AI models to complete complex tasks.

Unlike traditional systems that follow fixed workflows, agents dynamically adapt based on the situation. They can interpret a user's intent, assess the surrounding context, and make decisions on the user's behalf. As they operate, they learn from new information and refine their approach to better meet their objectives.

Agents are a common agentic AI implementation pattern, and the Model Context Protocol (MCP) is the open standard introduced to make tools, APIs, and data sources accessible to AI models. You'll learn all about agents and MCP in Part V so you can start building your own agentic AI solutions.

Plugging LLMs into Your Code

All of this is interesting, but just like our startup team, we need a way to plug these LLMs into our applications. This is where the rubber meets the road.

AI companies (such as OpenAI) and open source tools (such as Ollama) provide APIs and SDKs that make it easy for developers to call these AI models in our code. Figure 1-7 shows how ChatGPT uses these APIs to generate a response.

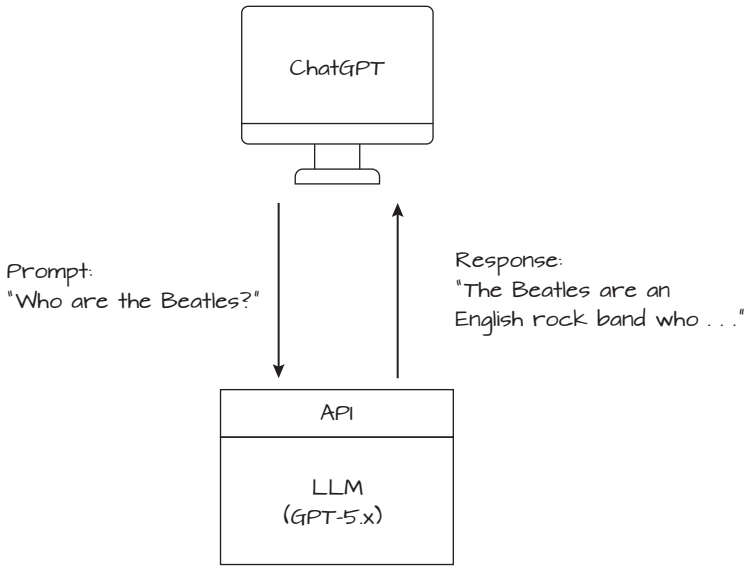


Figure 1-7: ChatGPT is an interface that uses OpenAI APIs to interact with LLMs.

As you can see, ChatGPT is really just an interface. When you send a prompt, ChatGPT is calling an OpenAI API to interact with the actual LLM generating the response.

In the upcoming chapters, we'll connect to a locally run, open weight LLM (Llama 3.2) through the Ollama APIs. We'll demonstrate through examples the use of low-level SDKs, lightweight wrappers, and high-level frameworks to interact with models. The Llama models are built by Meta and provided to the community as open weight models. This allows you to run the LLM locally without paying for usage.

You can use the same code and tools you'll find in this book to experiment with any pretrained LLMs. As you'll see in Chapter 4, the concepts remain the same, but your testing may reveal that a specific model provides better responses or performance, or is more cost-effective.

OPEN SOURCE VS. OPEN WEIGHT MODELS

As developers, we're very familiar with open source software whose code is freely available to inspect, modify, and share. It's the foundation of countless tools we use daily, from operating systems like Linux to libraries on GitHub that power nearly every modern application. We'll use several open source libraries to integrate AI models throughout this book.

In contrast, the AI community refers to models like Llama 3.2 as *open weight*, meaning the model's trained parameters are available, but the full source code, dataset, and training process may not be. This means that Llama 3.2 is not truly open source. Keep in mind that the license also restricts redistribution and certain types of commercial use. We'll revisit this topic in Chapter 4.

Summary

This chapter provided a high-level introduction to LLMs and general AI concepts. You started by learning how LLMs are trained on large datasets of publicly available and licensed text including books, articles, and research papers in order to understand natural language.

You then saw how LLMs generate text by predicting a sequence of tokens. You learned how AI models differ from traditional programming in that their outputs (predictions) are probabilistic, not deterministic. This gives them the flexibility to adapt to new situations but runs the risk of mistakes.

You saw the versatility of LLMs and the variety of use cases they're able to handle through prompting, including writing, summarizing, classifying, translating, coding, and much more. You were also introduced to the limitations that have to be considered and addressed when using an LLM, such as limited training knowledge, hallucinations, model bias, and challenges with handling multi-step reasoning.

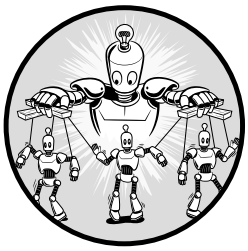
Finally, we introduced some of the major topics we'll cover in the book: prompt engineering, context engineering, fine-tuning, and agents. You learned that we'll be interacting with LLMs through APIs and SDKs.

Now it's time to dive into the code so we can get our hands dirty. In Chapter 2, we'll start just like our startup team did by calling an LLM from code, using JavaScript. We'll introduce some basic concepts along the way. Then, in Chapter 3, we'll introduce Python as we port over the example from JavaScript and prepare you for the chapters to come.

So come on, let's get coding!

2

BUILDING YOUR FIRST LLM-POWERED APPLICATION



As with anything new, we have to start with the basics. This chapter will serve as your "Hello, World!" to calling an LLM API from code and getting a response.

By the end of this chapter, you will have the basic knowledge and code necessary to do the following:

- Capture input from a user and build a prompt
- Send that prompt to an LLM through an API
- Stream the LLM response back to the user

All the concepts and code will be explained along the way. If you want to download the example, you can clone the following project from GitHub: *https://github.com/jorshali/developers-guide-to-ai*.

The code for this chapter is available in these folders:

- Calling an LLM via an API: *part1/getting_started/section1*
- Streaming your LLM responses: *part1/getting_started/section2*
- Building a user interface (UI) with Vite and React: *part1/client*

Calling an LLM by Using an API

In Chapter 1, you saw how the ChatGPT interface calls back to an OpenAI API to interact with the underlying LLM. In this chapter, you will do the same, but using a locally running Llama instance (see Figure 2-1).

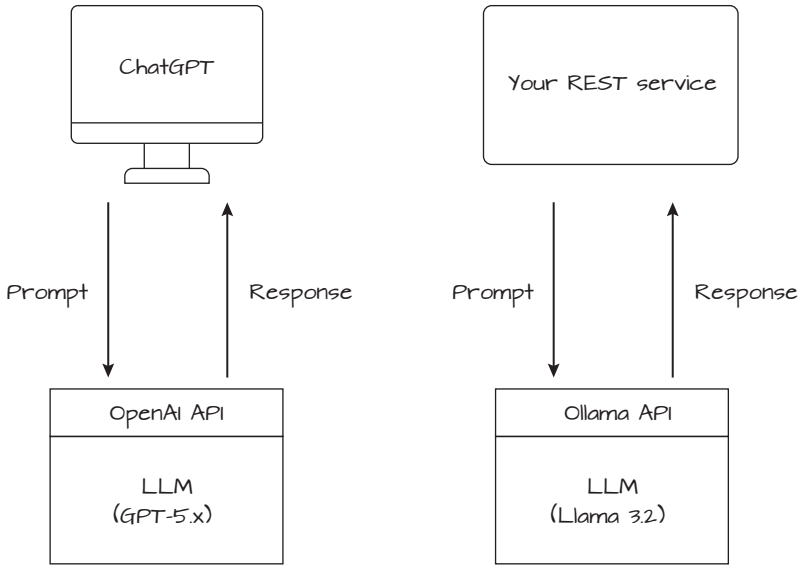


Figure 2-1: You will use an Ollama API (through an SDK) to call a local LLM.

You'll start by installing Ollama. This tool allows you to host your own LLMs locally. Then you'll build a simple Representational State Transfer (REST) service that combines Node.js, Express, and the Ollama JavaScript library to create a simple REST service that when called with a prompt will return an LLM response. Let's get coding!

Verify System Requirements

Before installing Ollama, it's worth noting that these models are large. It's in the name! You need to make sure your machine can handle the load.

Ollama uses *quantized models*. You'll learn more about these performance-optimized models and their trade-offs later (see "Dissecting Model Names" on page 138). For now, it's enough to understand that these are smaller model versions that trade a little accuracy for increased performance and reduced hardware requirements.

NOTE *You'll also see small language models (SLMs) used to describe lighter-weight models that run on modest hardware.*

Check your system specifications first. We'll be using Llama 3.2 3B throughout the text. This model requires 2GB of disk space to download and about 6GB of dedicated RAM to run.

If this is too much for your machine, you can substitute Llama 3.2 1B. This model requires 1.3GB of disk space to download and about 2GB of dedicated RAM to run. You will still be able to run the examples, although you might experience different results.

This is part of the fun! You'll see how models of different sizes and capabilities behave in a variety of scenarios.

Install Ollama and Run a Local LLM

Download and install Ollama from *https://ollama.com*. (All examples in this book were tested with Ollama 0.12.5.)

Once installed, Ollama automatically adds the Ollama command line interface (CLI) to your system path. This means you can run and manage models directly from your terminal without needing a separate application window.

You can verify the installation by running the following command in a terminal window (for example, Terminal in Mac or PowerShell in Windows):

```
~ % ollama --version
```

The Ollama CLI provides a simple way to download, run, and manage local models. Here are a few useful commands to get started:

`ollama serve` Starts the local Ollama server and lets you see live logs as your applications make requests to a local model

`ollama list` Lists all models currently downloaded and available on your system

`ollama pull` ***`model-name`*** Downloads the model identified by *`model-name`*

`ollama run` ***`model-name`*** Downloads (if necessary) and starts a local session with the model identified by *`model-name`*

Back in your terminal window, start by running the following command:

```
~ % ollama serve
```

You'll see the live logs for your Ollama server. Now you need a model. In a separate terminal window, execute the following command to download and run Llama 3.2 3B:

```
~ % ollama run llama3.2
```

Alternatively, execute the following command to download and run Llama 3.2 1B:

```
~ % ollama run llama3.2:1b
```

This may take a few minutes. Since this is your first time running a Llama 3.2 model, Ollama will automatically download it first.

Once the download completes and your model is running, a model session will be started:

```
% ollama run llama3.2
>>> Send a message (/? for help)
```

Here you can interact with the model, but first let's see which commands are available to us by executing the `/?` command:

```
>>> /?
Available Commands:
  /set            Set session variables
  /show           Show model information
  /load <model>   Load a session or model
  /save <model>   Save your current session
  /clear          Clear session context
  /bye            Exit
  /?, /help       Help \textbf{for} a command
  /? shortcuts    Help \textbf{for} keyboard shortcuts

Use """ to begin a multi-line message.
```

Don't worry if some of these commands don't make sense at the moment. They'll become clearer as you read on.

LM STUDIO IS A GUI-BASED OLLAMA ALTERNATIVE

Ollama is a CLI-first tool with a limited GUI for chatting. If you'd like to explore another option for running and experimenting with local models, check out LM Studio. Like Ollama, LM Studio lets you download, manage, and run open weight LLMs directly on your computer. It includes a clean graphical interface in addition to an API and command line tools.

LM Studio supports models from Hugging Face, Mistral, Llama 3, and others, and it automatically detects your hardware (CPU or GPU) to use the most efficient backend. You can chat with models, monitor performance, and even connect it to your own local applications via an OpenAI-compatible API endpoint.

In short, Ollama is lightweight and script-friendly, while LM Studio is great for developers who prefer a GUI-based workflow and easy model management. We will be using Ollama in the book, but you can learn more about LM Studio at *https://lmstudio.ai*.

Send a Prompt to the Local LLM

Once you have confirmed the installation, downloaded a model, and started a session, you can send a test prompt to Llama 3.2. Simply type **`Who are the Beatles?`** You should see a response similar to the following:

```
>>> Who are the Beatles?
The Beatles were a British rock band that formed in Liverpool, England in
1960. The band consisted of four members:
1. John Lennon (1940-1980) - rhythm guitar, keyboards, harmonica, and vocals
2. Paul McCartney (born 1942) - bass guitar, piano, keyboards, and vocals

--snip--
```

Now you need to set up a new project for your REST service. Make sure to keep Ollama running in the terminal window you already have open. You'll run the next commands in a new terminal window.

Install Node.js

If necessary, download and install Node.js from *https://nodejs.org*. (All examples in this book were tested with Node.js 24.8.0.)

You can verify the installation in a terminal as follows:

```
~ % node -v
```

If the installation was successful, the version will print.

Initialize Your Project

Create a new directory for the project (for example, *ai-for-devs*). Navigate to the new directory in a terminal and run the following command:

```
~/ai-for-devs % npm init -y
```

This creates a new *package.json* file, initializing the Node.js project.

Install Node.js Packages

Now you need to install the Node.js packages (library bundles) necessary to get your REST API up and running quickly. You will be using these JavaScript libraries:

`express` Makes server creation quick and easy

`ollama-js` Is the easiest way to integrate with Ollama

In a terminal, make sure you're still in your project folder (for example, *ai-for-devs*). Run the following `install` command:

```
~/ai-for-devs % npm install express@5.1.0 ollama@0.6.2
```

NOTE *Our examples were tested with the library versions shown here. This will be a common pattern throughout the book to ensure reliability, as libraries can change over time. The book's GitHub repository will be continually maintained with the latest and greatest library versions.*

Once this command completes, your *package.json* will list these packages as project dependencies (and a *package-lock.json* will record exact versions). Notice that a *node_modules* folder was created in your project directory. This folder contains the installed packages. There are quite a few! These are your project's dependencies and all *transitive dependencies*. Just as your project depends on `express` and `ollama`, these packages have dependencies of their own. These transitive dependencies are automatically installed for you by `npm` (the Node Package Manager).

Software developers stand on the shoulders of giants. We regularly reuse open source and proprietary libraries to help us build software quickly. The *package.json* file has many purposes, but its dependencies list tells `npm` which libraries your project needs (along with their versions) in order to function. It's like a list of ingredients in a recipe when you are cooking. As long as we have the recipe, we can always go and get the ingredients. In this way, we don't need to include our dependencies (the *node_modules* directory) when we share our project (for example, through Git).

Create the Server File

Create a file named *server.mjs* in your project directory. Open this file in a text editor or your favorite integrated development environment (IDE) such as Visual Studio Code (VS Code) and add the following lines of code:

```
❶ import express from "express";
  import { Ollama } from "ollama";

❷ const app = express();

❸ const ollama = new Ollama();

❹ app.get('/', async (request, response) => {
  ❺ response.type('text/plain');

  ❻ const modelResponse = await ollama.generate({
      model: 'llama3.2',
      prompt: "Can you simply say 'test'?"
    });

    response.send(modelResponse.response);
  });

❼ app.listen(8000, () => {
   console.log(`Server is running on port 8000`);
  });
```

This is a simple Express server that listens on port 8000 for any HTTP `GET` requests ❼. `GET` is the most basic HTTP request. When you load a website from a URL, the browser sends a `GET` request to initiate the page load.

The example starts by importing the following from the Node.js packages you installed ❶:

`express` The top-level function exported by the `express` module. It's essentially the constructor for an Express application instance, and it provides a way to manage routes, handle HTTP requests and responses, and use middleware.

`Ollama` The `ollama-js` library class that facilitates interaction with the Ollama API. This class makes it easy to integrate and utilize Ollama-hosted models within your JavaScript applications.

The `express` function is then used to initialize your application, and the `app` constant is set with the result ❷. An `Ollama` instance is created and set to the `ollama` constant for use in our service ❸.

The `get` method on the `app` instance is then used to configure the *route handler* function you want called when any `GET` request is received on the root URL (`'/'`) ❹. Notice that the route handler function receives `request` and `response` parameters:

`request` An object that represents the incoming HTTP request from the client. This object contains properties and methods that provide information about the request (such as headers, parameters, and body).

`response` An object that represents the HTTP response that the server will send back to the client. This object contains methods for sending data, setting headers, and managing the response process.

Within your route handler function, you start by telling the client the type of data that you will be sending back. In this case, you will be sending plain text ❺. The `response.type` method provides a convenient way to set the `Content-Type` header to `text/plain` in the response.

The `Content-Type` header is important for both client and server. This special header indicates the type of data being sent in a request or response (known as the media type or MIME type) so it can process that data properly. The data could be plain text, HTML, XML, JSON, and more. In the example, you're helping a browser optimize its rendering by telling it to expect plain text. There's no need to interpret HTML.

Now you can call the `generate` method on the `ollama` instance with the model you are using ❻ (in this case, `llama3.2`) and a simple `prompt`, **`Can you simply say 'test'?`** The `generate` method executes asynchronously, so you `await` the response from the LLM. Once the complete response is generated, it is sent back to the caller with the `response.send` method.

Figure 2-2 shows what the `Ollama` component from `ollama-js` is doing behind the scenes when a client calls your REST service.

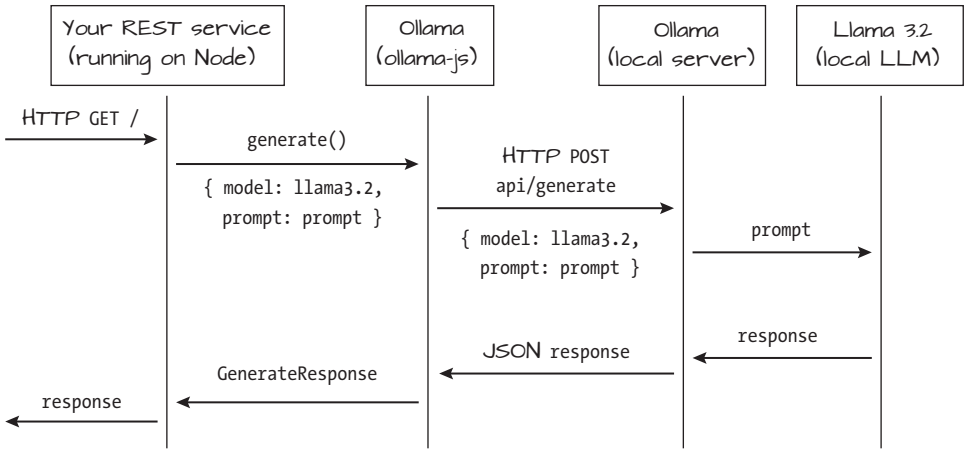


Figure 2-2: The sequence of invocations to prompt an LLM when your service is called

Now let's try it out!

Launch Your Server and Test

In the terminal, run this command:

```
~/ai-for-devs % node server.mjs
```

If your server starts successfully, you will see the following output:

```
Server is running on port 8000
```

Open your web browser and visit *http://localhost:8000*. You'll see a simple response in your browser window from the Llama 3.2 model: `Test`.

You have now created a basic REST service that calls a Llama model running on your local machine. Now we'll look at how to stream LLM responses to make these API calls feel significantly faster.

Streaming Your LLM Responses

The cool typing effect you see in the responses from ChatGPT makes it feel responsive. Without it, the UI would feel slow and often unbearable.

OpenAI ingeniously transformed a drawback of LLMs, their text-generation speed, into a captivating UI effect. Achieving a similar level of responsiveness is possible by employing LLM response streaming.

Remember that when you prompt an LLM, it generates a response token by token. Streaming captures these tokens as they are generated and sends them back to a caller in chunks. The size of the chunks will vary depending on the LLM, but this approach allows you to send generated text back to the user when it's ready. As your prompts and call chains grow in size and complexity, streaming can become crucial to performance (see Figure 2-3).

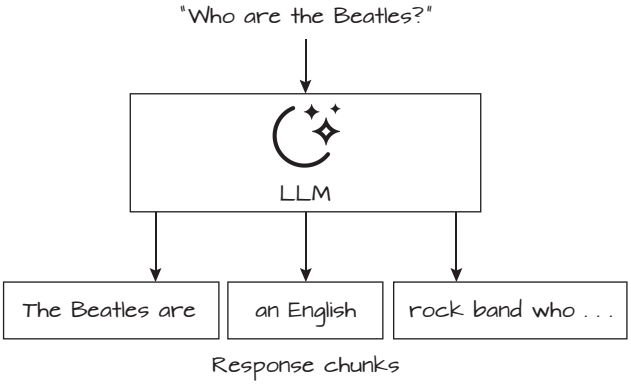


Figure 2-3: An LLM generating a response in chunks to improve perceived latency

Now that you have your REST service, you can build upon it to stream the LLM responses. You'll start by learning the APIs that make this possible with the Ollama JavaScript library and Express. Then you'll modify your *server.mjs* to stream back the LLM response.

Stream an LLM Response with Ollama

The Ollama JavaScript library provides the ability to stream any response. The following snippet shows how to specify that a response should be streamed as it's generated by the model:

```
import { Ollama } from "ollama";

const ollama = new Ollama();

const streamIterator = await ollama.generate({
  model: 'llama3.2',
  prompt: "Can you tell me about the Beatles in 500 words or less?",
❶ stream: true
});

--snip--
```

Setting the `stream` option to `true` when calling the `generate` method allows you to receive the LLM response in chunks ❶. Now you need a way to send these chunks back to the client as they are received.

Stream a REST Response with Express

The Express `Response` object provides the `write` method. This method allows you to send a response to the client as it is generated. This is exactly what

you need. The following snippet shows how this API could be used to send a response chunk to the client as it's generated by the `stream` method:

```
--snip--

const ollama = new Ollama();

app.post('/', async (request, response) => {
  response.type('text/plain');

❶ const streamIterator = await ollama.generate({
    model: 'llama3.2',
    prompt: "Can you tell me about the Beatles in 500 words or less?",
    stream: true
  });

❷ for await (const chunk of streamIterator) {
  ❸ response.write(chunk.response);
  }

  response.end();
});
```

You are doing the same thing as before, but the `generate` method ❶ now provides an *iterator* that you can loop over to receive the chunks ❷. Using `await` on `streamIterator` causes the `for` loop logic to run every time a `chunk` is generated by the LLM.

Within the `for` loop ❸, the Express `Response` object's `write` method is used to send each chunk's `response` string back to the client as soon as it's ready. The loop continues, and the chunks are streamed back to the client until the entire model response is complete.

Notice that you are now accepting an HTTP `POST` request. In a moment, we'll see how this allows a client to send any question in the body of the request so we can generate a response from the LLM.

Install the CORS Middleware Module

In this section, you'll be calling the server from a browser client. If you've ever set up your own server and made requests to it from a browser client, you've likely encountered cross-origin resource sharing (CORS). CORS can cause requests to mysteriously fail even though it looks like you followed the example perfectly.

CORS, a security standard that browsers implement, uses special headers sent by the client and server to prevent malicious activity. You can learn more about CORS with a simple Google search.

Express provides the `cors` module to handle these CORS requirements for you. In your terminal, make sure you're still in your project folder (for example, *ai-for-devs*). Run the following `install` command:

```
~/ai-for-devs % npm install cors@2.8.5
```

WARNING *In all these examples, you will be defaulting to relaxed CORS settings so your client can call your server. We are relaxing these settings throughout the book for simplicity, but always be sure CORS is configured appropriately before moving to production.*

Create the Entire Server File with Express

If you're following along in the code, replace your *server.mjs* file with the following to stream the LLM response to a client:

```
import express from "express";
import { Ollama } from "ollama";
import cors from "cors";

const app = express();

❶ app.use(cors());
app.use(express.json());

const ollama = new Ollama();

app.post('/', async (request, response) => {
  response.type('text/plain');

❷ const body = request.body;

  const streamIterator = await ollama.generate({
    model: 'llama3.2',
❸ prompt: body.question,
    stream: true
  });

  for await (const chunk of streamIterator) {
    response.write(chunk.response);
  }

  response.end();
});

app.listen(8000, () => {
  console.log(`Server is running on port 8000`);
});
```

You start by setting up the necessary Express middleware ❶. Express middleware provides a simple way to add logic that makes changes to a

request and response before and/or after your code executes. For example, the `cors` middleware will automatically add the request and response headers necessary to handle CORS.

The `express.json` middleware automatically parses JSON from incoming requests. You are telling Express to expect JSON in the request body. The parsed data is made available in the `request.body` attribute ❷. For example, the following JSON could be sent as the request body to the server:

```
{
  "question": "Can you tell me about the Beatles in 500 words or less?"
}
```

The `express.json` middleware parses this so you can directly retrieve the `body.question` attribute value and send the question as the prompt ❸.

The rest of the code takes advantage of the APIs you just learned about to stream back the LLM response. Now let's start the server and test the streaming.

Launch Your Server and Test

Once again in the terminal, run the following command:

```
~/ai-for-devs % node server.mjs
```

If your server starts successfully, you will see this output:

```
Server is running on port 8000
```

Since your server now receives a `POST` request with the question in the body, you'll use `curl` for testing. In a terminal, run the following command:

```
~ % curl -N -X POST -H "Content-Type: application/json" -d \
  '{"question": "Can you tell me about the Beatles in 500 words or less?"}' \
  http://localhost:8000
```

You will see a streaming response in your console from the Llama 3.2 model.

You have modified your REST service to receive a message that is sent to a Llama model, and the response is streamed back to the caller. Now you just need a UI client so you can easily send prompts to your LLM!

Stream to the UI with Fetch

The example code provides a UI client that will send a query to your server and stream the response. Throughout the book, we will provide a simple client when needed and will reference important code snippets. This will allow you to stay focused on integrating pretrained AI models.

The client can always be found in the folder for the part of the book referencing it (for example, *part1/client*). The README file in the client folder

provides the instructions to get the example up and running, and then you'll see the LLM streaming in action.

The UI client uses JavaScript to render the response chunks as they are returned to the UI by using the `fetch` function:

```
--snip--

const url = 'http://localhost:8000';
❶ const response = await fetch(url);
❷ const reader = response.body.getReader();

--snip--
```

The `fetch` function allows you to call your server and read its response as a `ReadableStream` ❶. You simply call `getReader` on the body of the response so you can read each `chunk` as it is returned and stream it back to the user ❷.

Decode the Response Chunks and Render

The `fetchData` function renders the chunks of text as they are returned:

```
--snip--

const fetchData = async () => {
  const url = 'http://localhost:8000';
  const response = await fetch(url);

  const reader = response.body
 ❶ .pipeThrough(new TextDecoderStream())
    .getReader();

  while (true) {
    const { done, chunk } = await reader.read();

 ❷ renderResponseChunk(chunk);

    if (done) {
      return;
    }
  }
};

--snip--
```

The response chunks you receive are returned as a stream of bytes. To get the actual text, you can pipe the response through `TextDecoderStream` ❶.

You can see how the `renderResponseChunk` function renders the chunks to the user via React in the example code ❷.

Summary

You have successfully built a functional REST service that streams responses from a server to a client. Beyond its ability to prompt an AI and generate responses, this service forms the foundation for the upcoming chapters.

Next, we'll introduce some Python basics so you can understand the remaining examples in the book.

Exercises

1. Look at the `chat` method on the `Ollama` class. See if you can modify the example to call this method and pass the prompt as part of a message array. You'll learn more about the message array in upcoming chapters.
2. Try modifying the example to log the `chunk` object as the response is being streamed (use `console.log`). Make a request and watch the data being sent back from Ollama along with each `chunk`. These are the attributes of the `GenerateResponse` class. The purpose of this data will become clear as you continue reading.